

Управления процессами в операционных системах

Оглавление

Определение многозадачности

Состав процесса

Местоположение процесса

Обеспечение защиты

Системные вызовы

Состояние процессов (потоков)

Планирование задач

Критерии планирования и требования к алгоритмам

Алгоритмы планирования

Windows NT

UNIX

OS/2

Управление памятью

Определение многозадачности

Многозадачность - способ организации работы системы, при котором в ее памяти одновременно содержатся программы и данные для выполнения нескольких процессов обработки информации.

При этом должна обеспечиваться взаимная защита программ и данных, относящихся к различным задачам, а также возможность перехода от выполнения одной задачи к другой, т.е. переключение задач.

Понятие процесса

Важнейшей частью операционной системы, непосредственно влияющей на функционирование вычислительной машины, является подсистема управления процессами.

Процесс (задача) — абстракция, описывающая выполняющуюся программу. Для операционной системы процесс представляет единицу работы, заявку на потребление системных ресурсов.

Подсистема управления процессами

Подсистема управления процессами

- ✓ планирует выполнение процессов,
- ✓ создает и уничтожает процессы,
- ✓ обеспечивает процессы необходимыми системными ресурсами,
- ✓ поддерживает взаимодействие между процессами.

Подсистема управления процессами

Для этого в памяти поддерживаются специальные информационные структуры, в которые система записывает, какие ресурсы выделены каждому процессу.

Подсистема управления процессами

Можно назначить процессу ресурсы:

- ✓ в единоличное пользование или
- ✓ в совместное пользование с другими процессами.

Ресурсы могут выделяться процессу:

- ✓ при его создании,
- ✓ динамически, по запросам во время выполнения.

Ресурсы могут быть приписаны процессу

- ✓ на все время его жизни
- ✓ на определенный период.

Подсистема управления процессами

При выполнении этих функций подсистема управления процессами взаимодействует с другими подсистемами операционной системы ответственными за управление ресурсами:

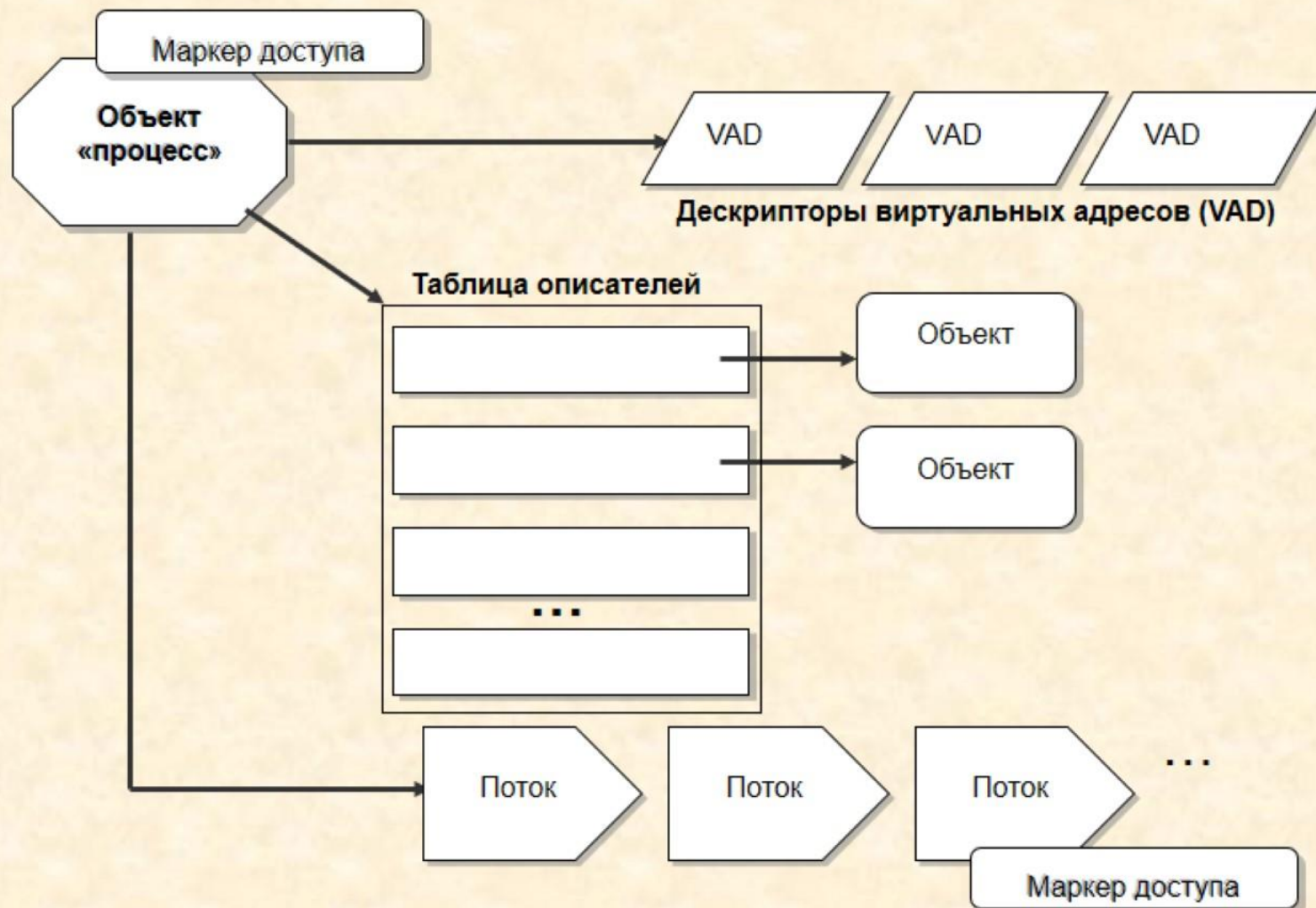
- ✓ управление памятью,
- ✓ подсистема ввода-вывода,
- ✓ файловая система.

Подсистема управления процессами

В ОС, где существуют процессы и потоки, процесс рассматривается как заявка на потребление всех видов ресурсов, кроме процессорного времени.

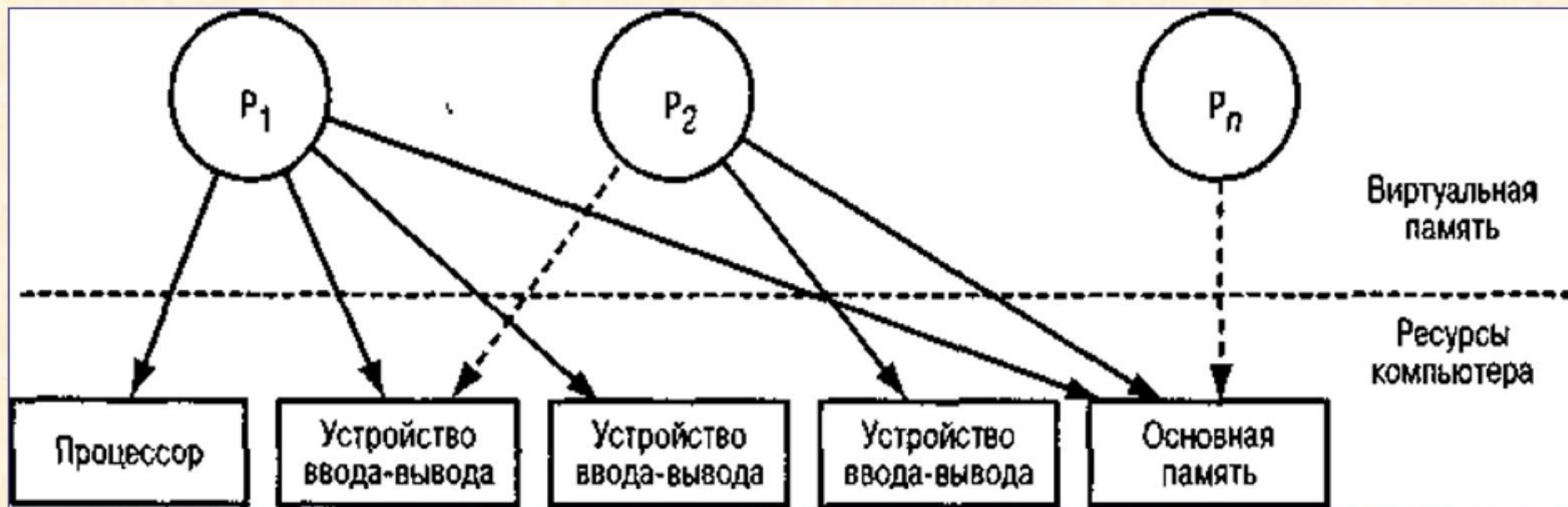
Процесс – это контейнер для набора ресурсов, используемых потоками. Процессорное время распределяется между потоками, которые представляют собой последовательности команд (потоки выполнения).

Состав процесса

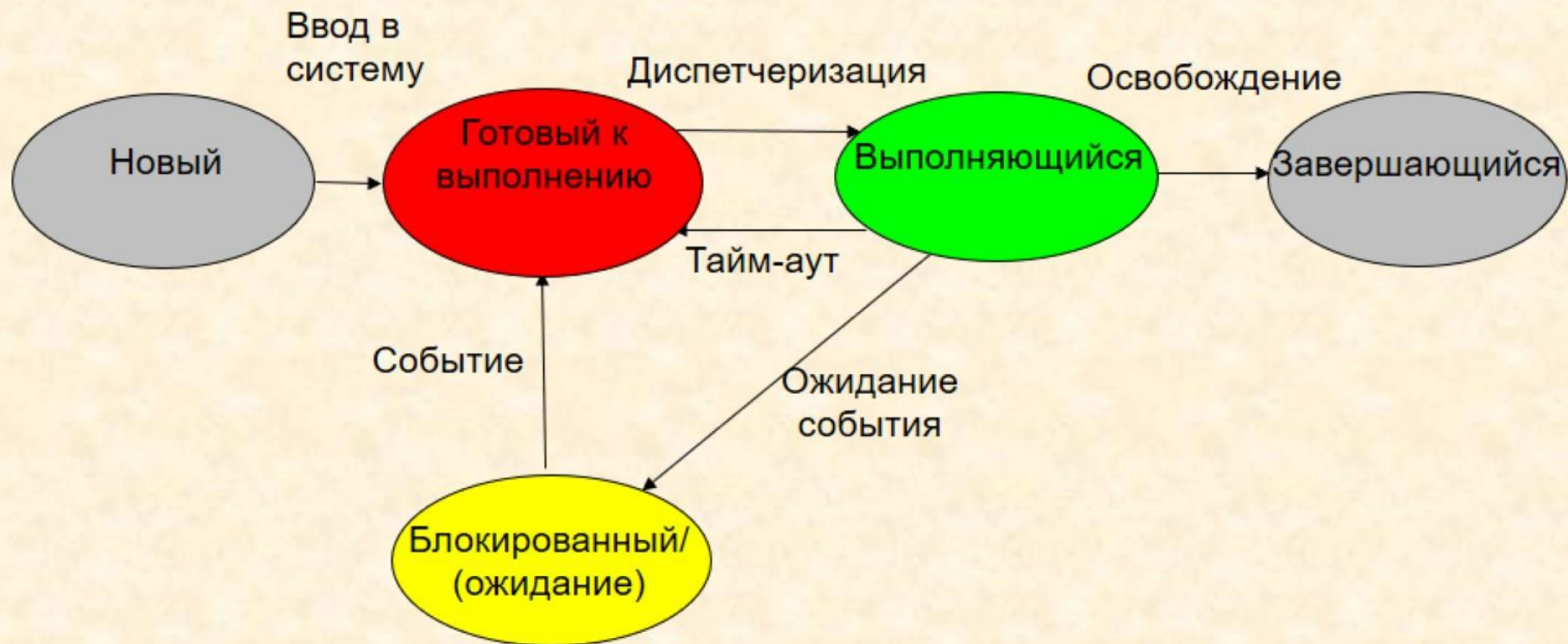


Операционная система - механизм, управляющий тем, как процессы используют системные ресурсы

Процессы и ресурсы в некоторый момент времени



Состояние процессов (потоков)



Планирование задач

Типы планирования

Цель планирования процессора состоит в распределении во времени процессов, выполняемых процессором таким образом, чтобы удовлетворять требованиям системы, таким, как время отклика, пропускная способность и эффективность работы процессора.

Типы планирования

Во многих системах планирование разбивается на три отдельные функции:

- ✓ долгосрочного планирования;
- ✓ среднесрочного планирования;
- ✓ краткосрочного планирования.

Их названия соответствуют временным масштабам выполнения этих функций. Функции планирования привязаны к диаграмме переходов состояния процесса.

Типы планирования

Долгосрочное планирование осуществляется при создании нового процесса и представляет собой решение о добавлении нового процесса к множеству активных в настоящий момент процессов в случае освобождения ресурсов памяти.

Типы планирования

Таким образом, долговременный планировщик решает, какие процессы будут выполняться одновременно, тем самым контролируя степень параллелизма и пропорцию между процессами, интенсивно выполняющими ввод-вывод, и процессами, интенсивно использующими процессор.

Типы планирования

Обычно в настольных компьютерах не применяется долговременный планировщик и новые процессы допускаются к выполнению автоматически. Но данный планировщик очень важен для систем реального времени, так как при чрезмерной нагрузке системы параллельно выполняющимися процессами время отклика системы может стать больше требуемого, что недопустимо.

Типы планирования

Среднесрочное планирование является частью свопинга (временное перемещение, или выгрузка процессов из основной памяти во вторичную например, на жёсткий диск, и наоборот) и представляет собой решение о добавлении процесса к множеству, по крайней мере, частично расположенных в основной памяти и, следовательно, доступных для выполнения процессов.

Типы планирования

Среднесрочный планировщик может принять решение выгрузить процесс из основной памяти если:

- процесс был неактивен некоторое время;
- процесс имеет низкий приоритет;
- процесс часто вызывает ошибки страниц;
- процесс занимает большое количество основной памяти, а системе требуется свободная память для других целей (например, чтобы удовлетворить запрос выделения памяти для другого процесса).

Типы планирования

Процесс будет возвращён в основную память, когда будет доступно необходимое количество свободной памяти или когда процесс выйдет из режима ожидания (в этом случае планировщик выгрузит из основной памяти другой процесс для освобождения основной памяти).

Типы планирования

Во многих современных системах, поддерживающих отображение виртуального адресного пространства на вторичную память, отличную от файла подкачки, среднесрочный планировщик может одновременно играть роль и долговременного планировщика, рассматривая новые процессы как процессы, которые были выгружены из основной памяти. Таким образом, система может подгружать в основную память программный код только тогда, когда он понадобится процессу для выполнения (это называется загрузкой по требованию или «ленивой загрузкой»)

Типы планирования

Краткосрочное планирование является решением о том, какой из готовых к выполнению процессов будет выполняться следующим.

Решения на этом уровне приходится принимать очень часто (как минимум, каждый временной отрезок). Также планировщик может поддерживать или не поддерживать вытесняющую многозадачность (то есть иметь возможность прервать исполнение какого-либо процесса).



Планирование потоков

Алгоритмы планирования могут преследовать различные цели и обеспечивать разное качество мультипрограммирования.

- ✓ ни один поток/процесс не будет занимать процессор дольше определенного времени,
- ✓ максимально быстрое выполнение «коротких» задач,
- ✓ равное время всем потокам (интерактивный режим).

Планирование потоков

Именно особенности реализации планирования потоков в наибольшей степени определяют специфику операционной системы, в частности, является ли она системой пакетной обработки, системой разделения времени или системой реального времени.

Планирование потоков

В большинстве операционных систем универсального назначения планирование осуществляется динамически (on-line), то есть решения принимаются во время работы системы на основе анализа текущей ситуации.

Планирование потоков

Другой тип планирования — статический — может быть использован в специализированных системах, в которых весь набор одновременно выполняемых задач определен заранее, например в системах реального времени.

Планирование потоков

Планировщик называется статическим (или предварительным планировщиком), если он принимает решения о планировании не во время работы системы, а заранее (off-line).

Планирование потоков

Результатом работы статического планировщика является таблица, называемая расписанием, в которой указывается, какому потоку/процессу, когда и на какое время должен быть предоставлен процессор.

Критерии планирования и требования к алгоритмам

Для каждого уровня планирования процессов можно предложить много различных алгоритмов. Выбор конкретного алгоритма определяется классом задач, решаемых вычислительной системой, и целями, которых необходимо достичь, используя планирование.

Критерии планирования и требования к алгоритмам

К числу целей планировщика можно отнести следующие:

- Справедливость – гарантировать каждому заданию или процессу определенную часть времени использования процессора в компьютерной системе, стараясь не допустить возникновения ситуации, когда процесс одного пользователя постоянно занимает процессор, в то время как процесс другого пользователя фактически не начинал выполняться.

Критерии планирования и требования к алгоритмам

К числу целей планировщика можно отнести следующие:

- Эффективность – постараться занять процессор на все 100% рабочего времени, не позволяя ему простаивать в ожидании процессов, готовых к исполнению. В реальных вычислительных системах загрузка процессора колеблется от 40 до 90%.

Планирование потоков

Смена активного потока происходит, если:

- поток завершился и покинул систему;
- произошла ошибка;
- поток перешел в состояние ожидания;
- исчерпан квант процессорного времени, отведенный данному потоку.

Планирование потоков

Поток, который исчерпал свой квант, переводится в состояние готовности и ожидает, когда ему будет предоставлен новый квант процессорного времени, а на выполнение в соответствии с определенным правилом выбирается новый поток из очереди готовых.

Кванты, выделяемые потокам, могут быть одинаковыми для всех потоков или различными.

Алгоритмы планирования

- Дисциплина **FCFS** – простейшая дисциплина диспетчеризации, согласно которой задачи обслуживаются в порядке их появления.
- Задачи, которые были заблокированы в процессе работы, после перехода в состояние готовности или для них организуется новая очередь, задачи из которых выполняются раньше тех, которые еще не выполнялись.

Алгоритмы планирования

FCFS не требует внешнего вмешательства в ход вычисления

Достоинства:

- простота реализации
- малые расходы системных ресурсов на формирование очереди задач
- нет бесконечного откладывания выполнения процесса

Алгоритмы планирования

- Дисциплина **SJN** требует, чтобы для каждого задания была известна оценка потраченного времени.
- Необходимость сообщать операционной системе характеристики задач, в которых описывались бы потребности ресурсов вычислительной системы, привела к тому, что были разработаны соответствующие языковые средства – язык управления задачами.

Алгоритмы планирования

Дисциплина **SJN** предполагает то, что короткие процессы выполняются раньше длинных.

При использовании дисциплины **SJN** операционная система ведет подсчет реальных потребностей и сравнивает заказанное время и время выполнения, и в случае превышения указанной оценки в данном ресурсе – задание становится в конец очереди.

Алгоритмы планирования

- В дисциплине **SRT** первым на выполнение становится поток, у которого осталось минимальное время до его завершения. Для того, чтобы проверить оставшееся время, нужен тот же язык управления. Время устанавливает сам программист.

Алгоритмы планирования

- Дисциплина RR предполагает, что каждая задача получает квант времени, по истечении которого она снимается с процессора, и процессорное время передается следующей задаче. Снятая задача ставится в конец очереди задач, готовых к выполнению.

Алгоритмы планирования

Для оптимальной работы системы необходимо правильно выбрать закон, по которому кванты времени выделяются задачам.

Величина кванта времени выбирается как компромисс между приемлемым временем реакции системы на запросы пользователей и накладными расходами на частую смену контекста задачи

Алгоритмы планирования

Планирование с использованием многоуровневой очереди (Multilevel queue scheduling). Эта стратегия разработана для ситуации, когда процессы могут быть легко классифицированы на несколько групп, например часто процессы разделяют на две группы: интерактивные (процессы переднего плана) и пакетные (фоновые).

Алгоритмы планирования

Интерактивные и пакетные процессы имеют различные требования к краткосрочному планировщику, например по отношению ко времени отклика.

Алгоритмы планирования

Стратегия многоуровневой очереди разделяет очередь готовых процессов на несколько очередей, в каждой из которых находятся процессы с одинаковыми свойствами, и каждый из которых может планироваться индивидуальной стратегией, например Round Robin стратегия для интерактивных процессов и FCFS для пакетных процессов.

Алгоритмы планирования

Взаимодействие очередей осуществляется по следующим правилам: ни один процесс с более низким приоритетом не может быть запущен, пока не выполнятся процессы во всех очередях с более высоким приоритетом.

Работа процесса из очереди с более низким приоритетом может быть приостановлена, если в одной из очередей с более высоким приоритетом появился процесс.

Алгоритмы планирования

Использование многоуровневой очереди с обратными связями (multilevel feedback queue scheduling).

Обычная многоуровневая очередь не допускает перемещения процессов между очередями. Многоуровневая очередь с обратными связями предполагает, что процессы при определенных условиях могут перемещаться между очередями.

Алгоритмы планирования

Многоуровневая очередь с обратными связями предполагает, что процессы при определенных условиях могут перемещаться между очередями. Здесь организуется N очередей. Все новые запросы поступают в конец первой очереди. Первый запрос из i -й очереди поступает на обслуживание лишь тогда, когда все очереди от 1-й до $i-1$ -й пустые.

Алгоритмы планирования

На обслуживание выделяется квант времени t_k . Если за это время обслуживание запроса завершается полностью, то он покидает систему. В противном случае недообслуженный запрос поступает в конец $i+1$ -й очереди



Алгоритмы планирования

После обслуживания запроса из i -й очереди система выбирает для обслуживания запрос из непустой очереди с самым младшим номером. Таким запросом может быть следующий запрос из очереди i или из очереди $i + 1$ (при условии, что после обслуживания запроса из очереди i последняя оказалась пустой). Новый запрос поступает в 1-ю очередь ($i = 1$).

Алгоритмы планирования

В такой ситуации после окончания времени t_k , выделенного для обслуживания запроса из очереди i , будет начато обслуживание запроса первой очереди. Если система выходит на обслуживание заявок из N -й очереди, то они обслуживаются либо по дисциплине FIFO (каждая заявка обслуживается до конца), либо по циклическому алгоритму.

Алгоритмы планирования

Данная система наиболее быстро обслуживает все короткие по времени обслуживания запросы.

Недостаток системы заключается в затратах времени на перемещение запросов из одной очереди в другую.

Данная стратегия является универсальной и сочетает в себе свойства всех рассмотренных раньше стратегий — FCFS, SJF, приоритетная, Round Robin, многоуровневая очередь.

Алгоритмы планирования

Приоритетная многоочередная дисциплина обслуживания.

Вновь поступающие в систему запросы устанавливаются не обязательно в 1-ю очередь, а в очередь в соответствии с имеющимися приоритетами, которые определяются параметрами обслуживания процессов.

Алгоритмы планирования



Алгоритмы планирования

Приоритетные многоочередные дисциплины обслуживания могут использовать обслуживание с абсолютным и относительным приоритетом. При обслуживании с абсолютным приоритетом приоритет определяется номером очереди, и первыми обслуживаются запросы, обладающие наивысшим приоритетом (из очереди с меньшим номером запрос из очереди $i - 1$ будет прерывать обработку запроса из очереди i)

Алгоритмы планирования

В данной дисциплине еще более увеличивается степень дискриминации по среднему времени ожидания в очереди между высоко- и низкоприоритетными запросами. Время ожидания высокоприоритетных заявок сокращается, но за счет большей задержки в обслуживании низкоприоритетных заявок. Достигается это за счет усложнения логики системы, дополнительной обработки запросов и выбора правила дообслуживания прерываемых процессов.

Алгоритмы планирования

- Приоритетное обслуживание предполагает наличие у потоков некоторой, изначально известной характеристики, на основании которой определяется порядок их выполнения.
- В большинстве ОС, поддерживающих потоки, приоритет потока непосредственно связан с приоритетом процесса, в рамках которого выполняется данный поток.

Алгоритмы планирования

- Приоритет процесса назначается операционной системой при его создании. Значение приоритета включается в дескриптор процесса и используется при назначении приоритета потокам этого процесса.

Алгоритмы планирования

- Изменения приоритета могут происходить по инициативе самого потока, когда он обращается с соответствующим вызовом к ОС или по инициативе пользователя, когда он выполняет соответствующую команду.

Алгоритмы планирования

Существует две разновидности приоритетного планирования:

- обслуживание с относительными приоритетами;
- обслуживание с абсолютными приоритетами.

Алгоритмы планирования

В обоих случаях выбор потока на выполнение из очереди готовых осуществляется одинаково, но проблема определения момента смены активного потока решается по-разному:

в системах с относительным приоритетом поток выполняется до тех пор, пока не закончится квант времени или пока он сам не покинет процессор, перейдя в состояние ожидания;

Алгоритмы планирования

В системах с абсолютным приоритетом выполнение активного потока может прерваться еще и в случае, если в очереди готовых потоков появился поток, приоритет которого выше приоритета активного потока.

В этом случае прерванный поток переходит в состояние готовности.

Windows NT

Данная ОС поддерживает 32 уровня приоритетов. Потоки делятся на два класса: реального времени и переменного приоритета.

Потоки реального времени, имеющие приоритеты от 16 до 31 - это высокоприоритетные потоки, используемыми программами с критическим временем выполнения, то есть требующие немедленного внимания системы.

Windows NT



Windows NT

Диспетчер задач просматривает очереди, начиная с самой приоритетной. При этом если очередь пустая, то есть, нет готовых к выполнению задач с таким приоритетом, осуществляется переход к следующей очереди. Следовательно, если есть задачи, требующие процессор немедленно, они будут обслужены в первую очередь. Для собственно системных модулей, функционирующих в статусе задач, зарезервирована очередь с номером 0.

Windows NT

- Большинство потоков в системе относятся к классу переменного приоритета с уровнями приоритета от 1 до 15.
- Эти очереди используются потоками с переменным приоритетом, так как диспетчер задач корректирует их приоритеты по мере выполнения задач для оптимизации отклика системы.

Windows NT

- Диспетчер приостанавливает исполнение текущего потока после того, как тот израсходует свой квант времени.
- При этом диспетчер задач понижает его приоритет на единицу и перемещает в другую очередь. Таким образом, приоритет потока, выполняющего много вычислений, постепенно понижается (до значения его базового приоритета).

Windows NT

- С другой стороны, диспетчер повышает приоритет потока после освобождения задачи (потока) из состояния ожидания. Величина добавки зависит от типа события, которого ожидал заблокированный поток.

Windows NT

- Так, например, поток, ожидавший ввода очередного байта с клавиатуры, получает большую добавку к значению своего приоритета, чем процесс ввода/вывода, работавший с дисковым накопителем. Однако в любом случае значение приоритета не может достигнуть 16.

UNIX

- В этой ОС отсутствует понятие поток, планирование осуществляется на уровне процессов. Каждый процесс в зависимости от задачи, которую он решает, относится к одному из трех определенных в системе приоритетных классов:
 - к классу системных процессов,
 - к классу реального времени
 - к классу процессов разделения времени.

UNIX

- Процессы системного класса зарезервированы для ядра и используют стратегию фиксированных приоритетов. Уровень приоритета процессу назначается ядром и никогда не изменяется.

UNIX

- Процессы реального времени используют стратегию фиксированных приоритетов, но пользователь может их изменять. Характеристики процессов реального времени включают две величины: уровень глобального приоритета и квант времени. Для каждого уровня приоритета по умолчанию используется своя величина кванта времени.

UNIX

- Процессы разделения времени используются пользовательскими задачами. Здесь используется стратегия динамических приоритетов.
- Величина приоритета, назначаемого процессу разделения времени вычисляется пропорционально значениям двух составляющих: пользовательской части и системной.

UNIX

- Пользовательская часть приоритета может быть изменена администратором и владельцем процесса (в последнем случае только в сторону уменьшения).

UNIX

- Система состояния позволяет планировщику управлять процессами в зависимости от того, как долго они занимают процессор, не уходя в состояние ожидания. У тех процессов, которые потребляют большие периоды процессорного времени без ухода в состояние ожидания, приоритет снижается.

UNIX

- У тех процессов, которые часто уходят в состояние ожидания после короткого использования процессорного времени, приоритет повышается.
- Понижение приоритета компенсируется тем, что процессам с низким приоритетом даются большие кванты времени, чем процессам с высоким приоритетом.

Управление памятью

Управление памятью

При рассмотрении различных механизмов и стратегий, связанных с управлением памятью, необходимо учитывать требования, которым они должны удовлетворять.

Требования к подсистеме управления памятью

- выделение блока памяти
- освобождение блока памяти
- изменение размера блока памяти выделяемой процессу
- обобщение блока памяти
- защита блока памяти выделенного процессу
- возможность совместного использования одной и той же области основной памяти совместными процессами.

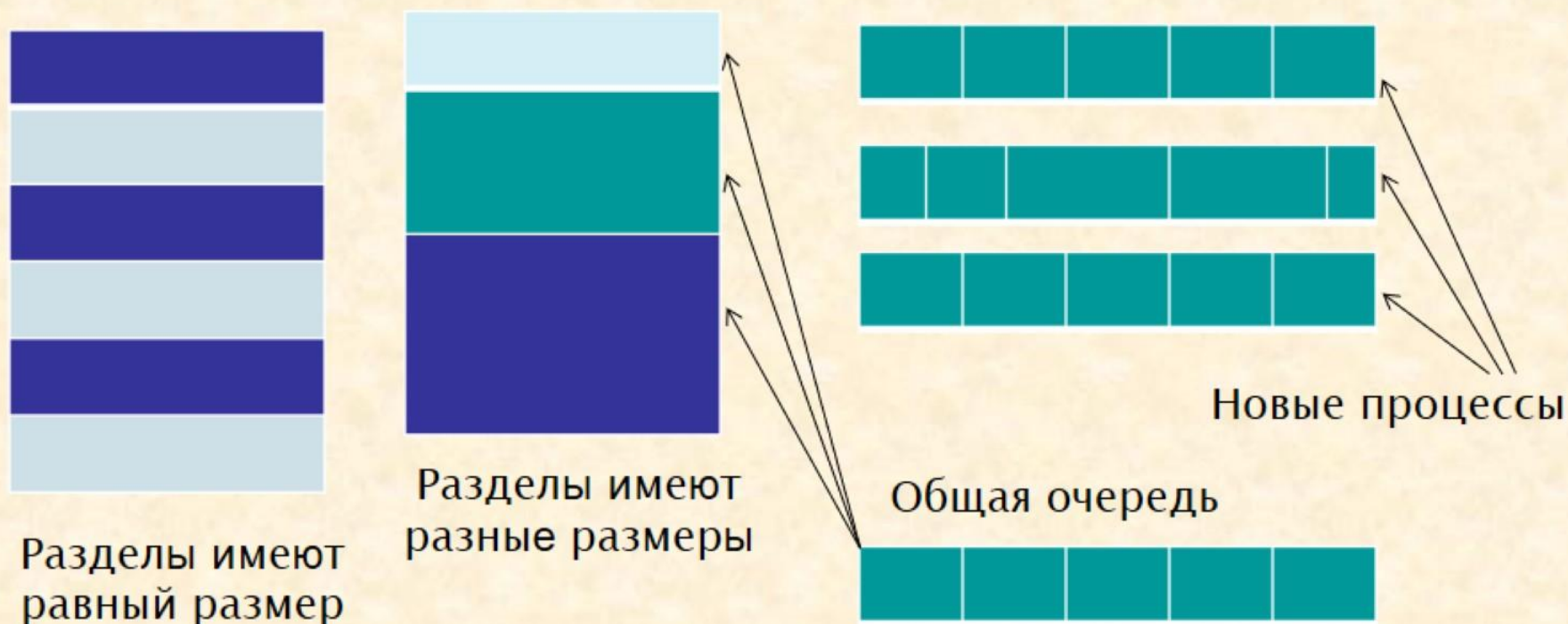
Основная память может быть распределена с использованием разных технологий:

Фиксированное распределение – это распределение, при котором основная память разделяется на ряд статических разделов во время генерации системы. Процесс может быть загружен в раздел равного или большего размера.

Фиксированное распределение

- Достоинство – простота реализации, малые системные накладные расходы.
- Недостаток – неэффективное распределение памяти.

Фиксированные разделы памяти



Динамическое распределение

Динамическое распределение – это распределение, при котором разделы создаются динамически, во время загрузки программы, т.е каждый процесс загружается в раздел строго необходимого размера.

Динамическое распределение

При динамическом распределении существует три основных алгоритма:

- первый подходящий
- самый подходящий
- самый неподходящий

Динамическое распределение

Метод «первый подходящий» проверяет все свободные блоки с начала памяти и выбирает первый достаточный по размеру для размещения процесса.

Динамическое распределение

Метод «самый подходящий» выбирает блок, размер которого наиболее близок к требуемому, в этом случае список блоков памяти рассортирован по возрастанию их размеров.

Динамическое распределение

Метод является наихудшим, так как в нем ищутся блоки, наиболее близкие по размеру к требуемому, а оставшиеся свободными блоки имеют очень маленький размер. Поэтому основная память очень быстро засоряется множеством мелких блоков.

Динамическое распределение

В методе «самый неподходящий» используется сортировка блоков памяти по убыванию их размеров, и процесс загружается в первый по списку блок; в оставшийся новый блок памяти может быть загружен еще один процесс.

Данный метод самый эффективный.

Динамическое распределение

К положительным сторонам можно отнести отсутствие фрагментации и более эффективное использование памяти.

Недостаток – неэффективное использование процессора из за необходимости противодействия внешней фрагментации (страницы физически не упорядочены).

Система двойников

В начале все доступное для распределения пространство рассматривается как единый блок. Если при запросе процессу требуется блок размером больше $\frac{1}{2}$ имеющегося, то выделяется весь блок, иначе блок делится на 2 эквивалентных двойника.

Система двойников

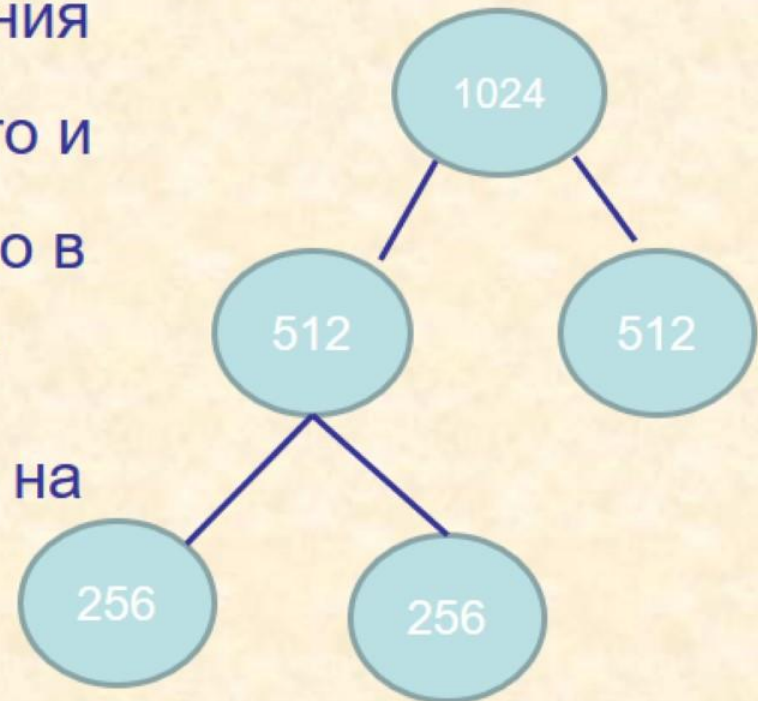
Если размер требуемой памяти больше $\frac{1}{2}$ размера нового блока, то один из двойников выделяется процессу, в противном случае один из двойников вновь делится пополам.

Система двойников

Этот процесс продолжается до тех пор пока не будет сгенерирован наименьший блок, размер которого не меньше требуемого.

Система двойников

Система двойников представляет собой компромисс для преодоления недостатков схем фиксированного и динамического распределения, но в современной ОС её превосходит виртуальная память, основанная на страничной организации и сегментации.



Виртуальная память

Виртуальная память - это совокупность программно-аппаратных средств, позволяющих пользователям писать программы, размер кода и данных которых превосходит имеющуюся оперативную память; для этого виртуальная память решает следующие задачи:

- ▶ размещает данные в запоминающих устройствах разного типа, например, часть программы в оперативной памяти, а часть на диске;
- ▶ перемещает по мере необходимости данные между запоминающими устройствами разного типа, например, подгружает нужную часть программы с диска в оперативную память;
- ▶ преобразует виртуальные адреса в физические.



Виртуальная память

